

AgInTi C12880MA 高性能固件与 SDK

STM32H743 采集链路逆向理解、clean-room 重构、协议与验证手册

lazying.art

2026 年 7 月 19 日 版本 0.2

安全边界

本文对应的是一份全新编写且仅完成编译验证的固件。本文生成过程中没有执行烧录、擦除、解除保护、Option Bytes 写入或 EEPROM 写入。现有 2 MiB 固件镜像能够在 SWD 仍可用时恢复内部应用 Flash，但它不是完整板卡快照；EEPROM 校准、Option Bytes、OTP 与保护状态仍需在首次烧录前分别只读备份。

摘要

本报告把 C12880MA 光谱仪控制板的现有接口行为转写为可审计的硬件契约，并据此设计一套新的 STM32H743 固件、二进制协议与 Python SDK。现有程序的核心瓶颈不是 C12880MA 光敏阵列本身，而是逐边沿 HAL GPIO 调用、逐像素 ADC 轮询、采集与发送串行化，以及每帧主机请求。新设计使用 TIM2 事件驱动 DMA 写 GPIOE_BSRR 产生传感器时钟，使用 ADC1 外部触发与 DMA 获取 294 个原始样本，并用两个非缓存帧槽把采集和 USB CDC 发送重叠。旧 590 字节协议仍被保留；新 V2 协议增加序号、时间戳、实际曝光、状态、丢帧计数和 CRC-32。本文明确区分二进制中已经验证的事实、新固件的工程设计，以及必须通过示波器、稳定光源和对照固件验证的项目。

目录

1	目标、范围与证据等级	4
1.1	工程目标	4
1.2	证据等级	4
2	恢复能力：现在能恢复什么	4
3	现有固件的硬件契约	5
3.1	引脚与外设	5
3.2	USB 并不是 USB High-Speed	6
3.3	现有采集序列	6
3.4	590 字节帧的准确含义	6
4	现有性能瓶颈	7
4.1	逐边沿函数调用	7

4.2	逐像素 ADC 轮询	7
4.3	串行化的数据路径	7
4.4	带宽预算	7
5	新固件总体结构	7
5.1	帧槽状态机	8
5.2	内存与 Cortex-M7 Cache	8
6	高性能采集后端	8
6.1	为什么不能直接把 PE3 设为定时器 PWM	8
6.2	DMA 采样相位	9
6.3	积分时间与长曝光	9
6.4	为何默认只设到 1 MHz	9
6.5	DIRECT 参考后端	10
7	协议设计	10
7.1	旧协议	10
7.2	V2 命令	10
7.3	V2 光谱帧	11
8	USB、队列与错误恢复	11
8.1	接收路径	11
8.2	发送路径	11
8.3	连续流式优于逐帧请求	12
9	EEPROM 的只读策略	12
10	Python SDK	12
11	构建方法与产物	12
12	首次烧录前的强制检查	13
13	首次上板的测量计划	13
13.1	阶段 A: 无光学结论的电气验证	13
13.2	阶段 B: 数据语义对照	13
13.3	阶段 C: 性能阶梯	14
14	性能上限的现实判断	14
14.1	传感器下限	14

14.2 1 kFPS 是否可行	14
15 尚未解决的问题	15
16 结论	15

1 目标、范围与证据等级

1.1 工程目标

本项目希望在不改变现有 PCB 传感器接线的前提下实现：

- 正确复现 C12880MA 的 ST、CLK、视频输出采样顺序；
- 以确定性硬件时序替代逐边沿函数调用；
- 以 DMA 替代逐像素 ADC 轮询；
- 采集下一帧时并行发送上一帧，降低帧间死区；
- 保留供应商软件所需的旧接口，并提供适合高速流式采集的新接口；
- 对每一帧记录足够的可诊断元数据，使高帧率结果可验证而不是只显示一个 FPS；
- 在首次烧录前保持原板卡完全不变。

1.2 证据等级

等级	含义
已验证	来自只读 SWD 识别、完整内部 Flash 备份、向量表、反编译控制流、字符串、工作中的串口/USB 协议，或多个证据彼此一致。
官方佐证	与 ST 或 Hamamatsu 官方数据手册一致，并能解释二进制行为。
新设计	clean-room 源码已经写出并能够编译，但尚未在目标板运行。
待验证	必须依赖示波器、逻辑分析仪、稳定光源、暗场或供应商固件对照。

已验证（Verified）

当前只读备份 SHA-256 为 67F1F6C421D56C2077D5A3F7417AA6F5213A2791D0C63AE5DAFBDBDF461764B4，大小为 2,097,152 字节。它覆盖 STM32H74x/75x 的 2 MiB 内部 Flash 空间。

2 恢复能力：现在能恢复什么

对象	当前状态	结论
内部 Flash 0x08000000–0x081FFFFFF	已完整备份并哈希	在 SWD 可用且保护状态允许时，可按字节恢复应用代码。
外部 I ² C EEPROM	尚未形成独立归档	含校正数据；首次烧录前必须通过现有固件只读导出。已知主要区域至少包括 0x0000–0x0FFF 与 0x1068 附近。

对象	当前状态	结论
Option Bytes / RDP / 启动配置	尚未形成清单	不属于普通 Flash 镜像；必须读取并记录，不得盲目写回。
OTP 与器件 UID	UID 可读，OTP 未归档	UID 只用于识别，不应公开；OTP 必须保持只读。
系统 Boot ROM	ST 出厂固化	不需要也通常不能作为普通应用镜像备份。

尚未硬件验证

因此准确表述是“内部 Flash 可恢复”，不是“整块板已完全可恢复”。只有在 EEPROM、Option Bytes 和必要 OTP 状态也完成只读归档后，才可以把恢复包称为板卡级备份。

3 现有固件的硬件契约

3.1 引脚与外设

表 3: 已恢复的信号映射

功能	STM32H743 资源	证据/说明
C12880MA ST	PE2	GPIO 推挽输出
C12880MA CLK	PE3	GPIO 推挽输出；没有可直接使用的定时器输出 AF
模拟视频	PA0 / ADC1_INP16	ADC1，单端，16 位配置
外部触发	PE9 / EXTI9_5	上升沿触发采集
模式输出	PE13、PE14	由命令模式字节控制
校准 EEPROM	PB6/PB7	地址 0xA0，16 位存储器地址
USB	PB14/PB15	OTG HS 控制器的 embedded FS PHY

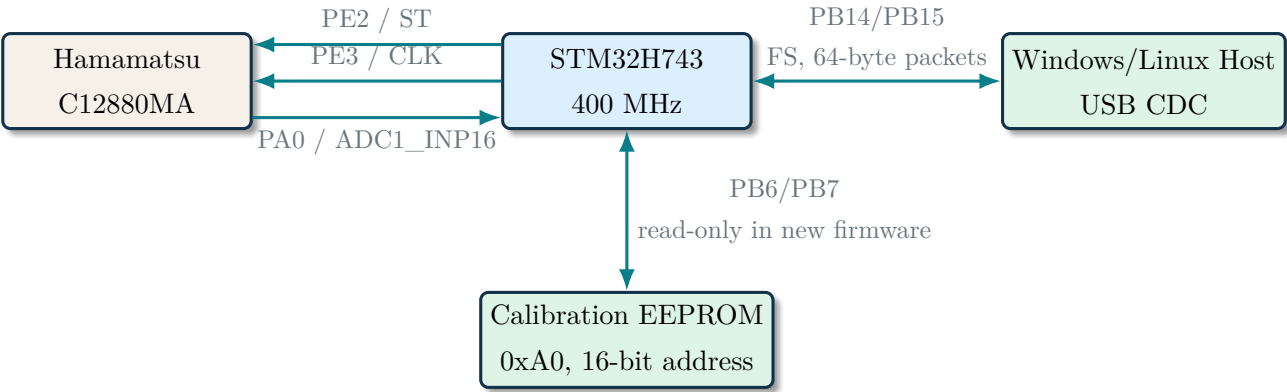


图 1: 控制板的最小硬件模型。箭头方向表示控制或数据方向。

3.2 USB 并不是 USB High-Speed

现有程序使用 USB_OTG_HS 控制器，但端点最大包长为 64 字节，PHY 配置是 embedded full-speed PHY。STM32H743 若要真正达到 480 Mbit/s，需要外部 ULPI PHY。因此“使用 HS 控制器”不能等价于“链路在 High-Speed 运行”。新固件保持与现有 PCB 一致的 Full-Speed 配置。

3.3 现有采集序列

二进制控制流给出的顺序如下：

1. 清零从 0x24000000 开始的帧缓冲区；
2. 输出 12 个预时钟；
3. PE2 (ST) 置高；
4. 输出命令指定数量的积分时钟；
5. PE2 (ST) 置低；
6. 输出 91 个 dummy 时钟；
7. 对 294 个样本逐个启动 ADC、轮询完成、读数，并在每个样本后输出一个时钟；
8. 发送 590 字节。

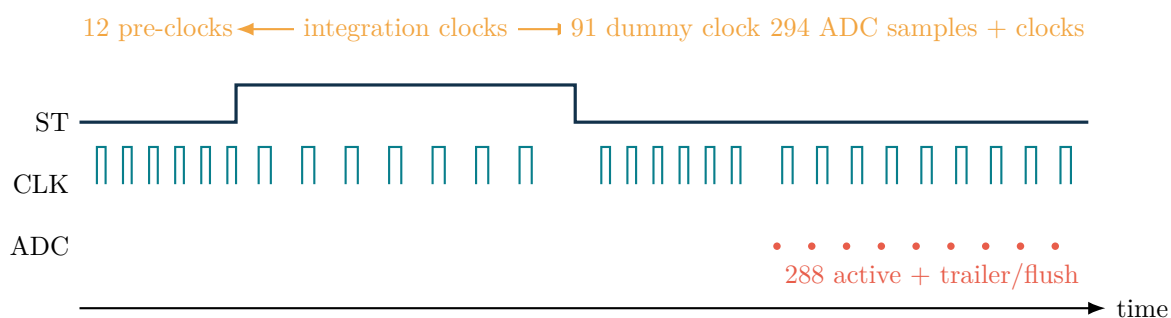


图 2: 现有固件的逻辑时序。图中脉冲为分段示意，不按真实脉冲数量绘制。

3.4 590 字节帧的准确含义

现有代码获取 294 个 16 位 ADC 值，并从帧缓冲区的第 6 个字开始存储。它只发送 295 个字，即：

$$590 \text{ bytes} = 6 \times 2 \text{ header} + 288 \times 2 \text{ active pixels} + 1 \times 2 \text{ trailer.}$$

最后 5 个已经采集的 flush 样本没有通过旧帧发送。V2 协议保留全部 294 个样本，便于验证读出相位、尾部稳定性和未来算法。

4 现有性能瓶颈

4.1 逐边沿函数调用

现有程序的传感器时钟并非由定时器输出，而是反复调用 GPIO 写函数。一些脉冲辅助函数甚至在一个电平上连续调用三次或四次写操作。这会引入函数调用、参数检查、分支和总线访问开销，也会使边沿间隔依赖编译结果与中断。

4.2 逐像素 ADC 轮询

每个样本都经历“启动 ADC、轮询 EOC、读取、停止/下一次”的软件序列。CPU 在转换期间没有准备下一帧、解析命令或构建 USB 数据。该模式易理解，但不适合作为极限性能路径。

4.3 串行化的数据路径

现有主循环近似为

$$T_{\text{frame, legacy}} = T_{\text{command}} + T_{\text{capture}} + T_{\text{send}} + T_{\text{wait}}.$$

新的双缓冲流式设计希望接近

$$T_{\text{frame, pipeline}} \approx \max(T_{\text{capture}}, T_{\text{send}}),$$

前提是主机持续读取、USB 队列不阻塞，并且下一帧在空闲帧槽中采集。

4.4 带宽预算

旧帧在 1000 frame/s 时的净载荷为

$$590 \times 1000 = 590 \text{ kB/s}.$$

旧固件中观察到的 1.5 Mbaud UART，在 8-N-1 下理论净字节率只有

$$\frac{1.5 \times 10^6}{10} = 150 \text{ kB/s},$$

相应帧率上限约为

$$\frac{150000}{590} \approx 254 \text{ frame/s}.$$

因此 1000 FPS 不可能依赖该 UART。USB Full-Speed CDC/Bulk 在持续读取时具有更高容量，但 Windows 驱动调度、短包、端点 FIFO 和主机读取粒度都会降低实际吞吐，必须用序号与丢帧统计实测。

5 新固件总体结构

新设计 (Designed, not flashed)

新工程位于 `firmware/sdk/`，包含可移植 C 核心、STM32H743 后端、USB CDC、旧协议兼容层、V2 协议、Python SDK、构建脚本和本文。工程中不存在烧录命令。

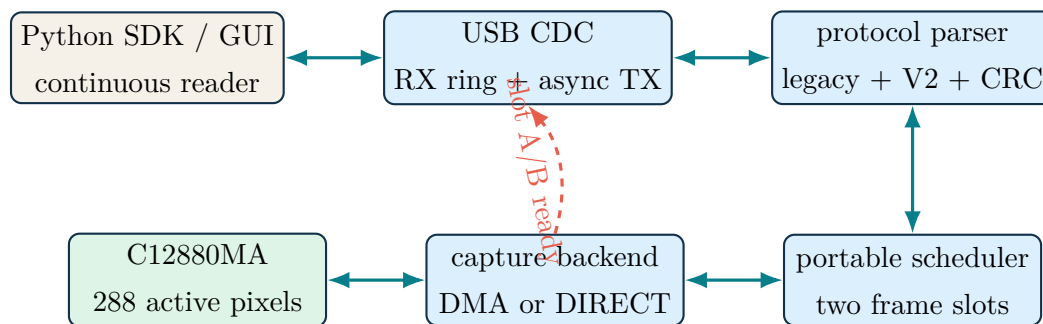


图 3: 新固件的软件分层。采集与发送通过帧槽状态而不是共享临时缓冲耦合。

5.1 帧槽状态机

每个帧槽有四个状态：

FREE → CAPTURING → READY → TRANSMITTING → FREE.

只有采集后端写 `raw[]`，只有封包阶段写 `tx[]`，USB 在 TRANSMITTING 期间只读 `tx[]`。状态转换在短临界区中进行，长时间采集和 USB 传输不关闭中断。

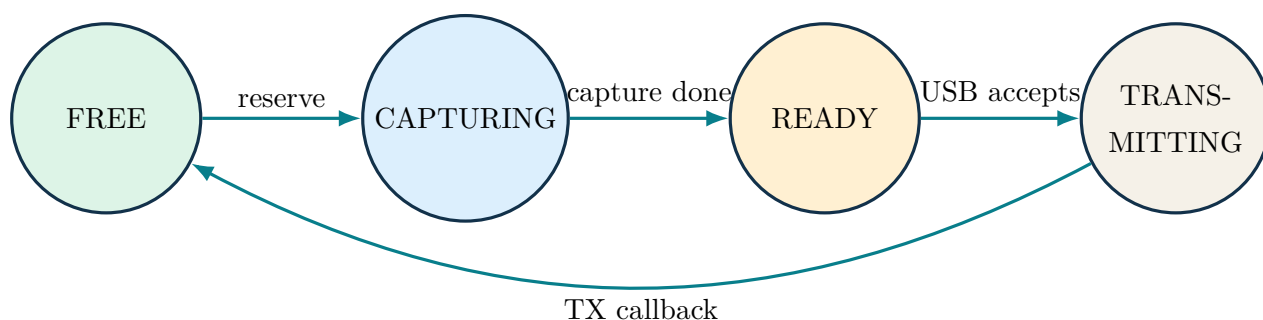


图 4: 双缓冲中单个帧槽的生命周期。

5.2 内存与 Cortex-M7 Cache

STM32H743 的 D-Cache 会造成“CPU 看到新数据但 DMA 看到旧数据”或相反的问题。新固件把整个设备对象、两个帧槽和 DMA 常量放在 D2 SRAM 的 `.dma_buffer` 段，并用 MPU 把 `0x30000000` 起始的 256 KiB 标记为 non-cacheable。这样牺牲少量 CPU 缓存收益，换取清晰且不易遗漏的 DMA 一致性规则。USB PCD 自身关闭 DMA，因此 USB 中间件缓冲不依赖额外 cache clean。

6 高性能采集后端

6.1 为什么不能直接把 PE3 设为定时器 PWM

STM32H743 数据手册给出的 PE3 复用功能不包含适合输出传感器 CLK 的 TIM 通道。因此在不改 PCB 的条件下，不能简单执行“TIMx PWM → PE3”。本设计把定时器事件作为 DMA request：

- TIM2 Update: DMA1 Stream1 把 `GPIO_PIN_3` 写入 `GPIOE_BSRR`，置高 PE3；
- TIM2 CH1 Compare: DMA1 Stream2 把 `GPIO_PIN_3 << 16` 写入同一 BSRR，置低 PE3；

- TIM2 CH2 的内部参考信号：作为 TIM2_TRGO，在低电平稳定区触发 ADC1。

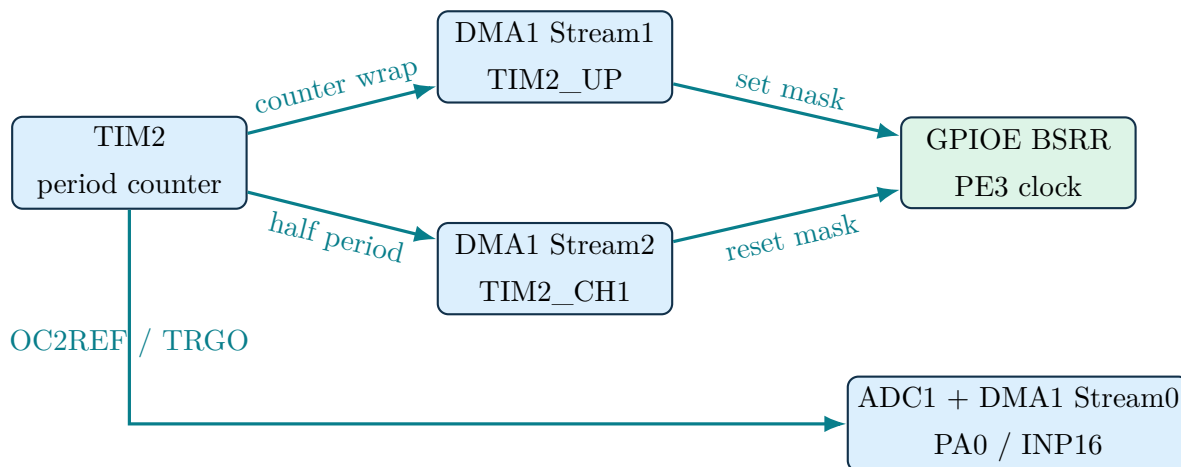


图 5: 在 PE3 没有定时器输出 AF 时，用定时器触发 DMA 写 BSRR 产生硬件时钟。

6.2 DMA 采样相位

旧程序是“91 个 dummy clock 后采第一个值，再输出时钟”。新 DMA 方案先输出 90 个 dummy clock，然后启动一个 294 周期的读出段。每个读出周期先产生第 91、92、...个时钟，在下降沿后的低电平稳定区触发 ADC。这样第一个 ADC 值仍对应第 91 时钟后的视频输出，且全部 294 点由同一硬件相位获取。

6.3 积分时间与长曝光

定义传感器时钟为 f_{clk} ，积分时钟数为 N_{int} ，则名义积分时间

$$T_{\text{int}} = \frac{N_{\text{int}}}{f_{\text{clk}}}.$$

在 1 MHz 下，1 个时钟为 1 μs 。在 2 MHz 下，1 个时钟为 0.5 μs 。DMA 的 NDTR 为 16 位；超过 65535 个时钟的曝光被分块。分块之间会存在很短的软件重启间隔，因此“时钟计数”仍准确，但真实曝光略长。若需要长曝光的绝对时间精度，应增加 32 位从定时器门控，而不是把该间隔隐藏。

6.4 为何默认只设到 1 MHz

C12880MA 数据手册允许最高 5 MHz CLK，但整个系统还包含 ADC 采样保持与 16 位转换。ADC 时钟、采样周期、转换周期和模拟前端建立时间必须满足：

$$T_{\text{sample}} + T_{\text{convert}} + T_{\text{margin}} < \frac{1}{f_{\text{clk}}}.$$

把传感器设到 5 MHz 并不自动意味着 16 位 ADC 可以以 5 MS/s 可靠输出。因此 clean-room 固件默认 1 MHz，软件上限暂定 2 MHz。只有对比暗场噪声、峰值位置、相邻像素串扰和 ADC overrun 后，才应逐级提高。

6.5 DIRECT 参考后端

DIRECT 后端保留逐样本流程，但直接使用 DWT 周期计数器稳定脉宽。它用于：

- 首次确认引脚极性和读出顺序；
- 与 DMA 后端进行逐像素差分；
- 当 DMA 相位有问题时提供故障隔离基线；
- 证明性能提升来自硬件流水而非改变光谱数据语义。

它不是性能目标。

7 协议设计

7.1 旧协议

身份请求为三个字节 04 0D 0A。兼容采集命令为 9 字节：

$\underbrace{\text{FF}}_{\text{start}} \quad \underbrace{\text{opcode}}_1 \quad \underbrace{\text{mode}}_1 \quad \underbrace{\text{exposure}}_{4, LE} \quad 0D \ 0A.$

正常采集返回固定 590 字节。EEPROM 读命令 0x09 返回 4096 字节，0x10 从 0x1068 附近读取 0x30 字节。为了防止校准被意外破坏，新固件不实现 0x08 和 0x11 的写行为。

7.2 V2 命令

表 4: 20 字节 V2 命令头

偏移	长度	字段
0	4	ASCII ASC2
4	1	协议版本 2
5	1	opcode
6	2	flags
8	4	sequence
12	2	payload length, 最大 64
14	2	reserved
16	4	CRC-32
20	N	payload

CRC 使用 IEEE CRC-32，计算时先把 CRC 字段置零。命令支持 HELLO、GET_CAPS、CONFIGURE、SINGLE_SHOT、START_STREAM、STOP_STREAM、GET_STATUS 和只读 EEPROM_READ。

7.3 V2 光谱帧

偏移	长度	字段
0	4	ASCII ASP2
4	1	version = 2
5	1	header bytes = 48
6	2	flags; bit 0 表示 little-endian samples
8	4	frame sequence
12	4	payload bytes, 通常 588
16	8	capture-complete timestamp, 单位微秒
24	4	integration clocks
28	4	实际 sensor clock, 单位 Hz
32	2	active pixels = 288
34	2	raw samples = 294
36	4	状态位, 包括 timeout、ADC overrun 与后端类型
40	4	累计 dropped frames
44	4	CRC-32
48	588	294 个 little-endian uint16 原始样本

序号可以检测丢帧；时间戳可以测量真实帧间隔；状态位可以区分“FPS 看起来高”与“ADC 已经 overrun”；CRC 可以检测串口缓存错位或不完整帧。

8 USB、队列与错误恢复

8.1 接收路径

CDC OUT 回调只把字节复制进 1024 字节单生产者/单消费者环形缓冲，并立即重新 arm 端点。命令解析在主循环执行，避免在 USB 中断中进行 EEPROM 访问或启动长采集。环形缓冲溢出会被计数并通过状态命令公开。

8.2 发送路径

帧数据生命周期由帧槽保障。USB 接受缓冲后，帧槽进入 TRANSMITTING；只有 CDC transmit-complete 回调后才回到 FREE。控制回复使用单独的 4096 字节缓冲，且与帧发送互斥。若主机不读数据，采集不会覆盖正在发送的帧。

8.3 连续流式优于逐帧请求

逐帧请求会把主机调度延迟叠加到每一帧。V2 的 `START_STREAM` 让设备在两个帧槽允许的范围内持续采集；主机只需要大块连续读取并按 `ASP2` 魔数重同步。停止命令不擦除数据，也不修改 EEPROM。

9 EEPROM 的只读策略

现有 EEPROM 使用 8 位地址 `0xA0/0xA1` 和 16 位 memory address。新固件用 `PB6/PB7` 实现低速、开漏、只读软件 I²C，原因是：

- 不依赖尚未示波验证的 I2C TIMINGR 常量；
- 读取校准发生在停止流式采集后，对吞吐没有影响；
- 源码中不存在 EEPROM 写函数，降低误操作风险。

首次在板验证时必须确认 SDA/SCL 上拉、电平和 ACK；在此之前“代码编译成功”不能证明 EEPROM 地址或引脚绝对正确。

10 Python SDK

Python 包 `aginti-c12880` 提供：

- V2 命令编码和 CRC；
- 增量流解码，可在丢字节后搜索 `ASP2` 重同步；
- 294 点原始样本和 288 点有效像素视图；
- 无后台线程的串行客户端，方便嵌入现有 GUI、Web API 或采集服务；
- CLI 身份检测、配置与持续流式 FPS/峰值输出。

```
from aginti_c12880 import SpectrometerClient

with SpectrometerClient("COM7") as dev:
    dev.configure(clock_hz=1_000_000, exposure_clocks=1000)
    dev.start()
    for frame in dev.frames(duration=5.0):
        spectrum = frame.pixels
        print(frame.sequence, frame.timestamp_us, spectrum.max())
    dev.stop()
```

11 构建方法与产物

```
./scripts/fetch_c12880_firmware_deps.ps1
./scripts/build_c12880_firmware.ps1 -Engine DMA -Configuration Release
```

依赖脚本固定以下公开源码提交：

- STM32H7 HAL: c5e70527126710a6415929ff10c1fd1f40394b1e;
- CMSIS Device H7: de8243d2c15f87936f28a49fcd9e6f5ba10fc233;
- CMSIS 5: 55b19837f5703e418ca37894d5745b1dc05e4c91;
- ST USB Device: 2df324bd60d4b0bb27404fd70b1c089b467f0e09。

构建生成 ELF、BIN、HEX、MAP, 并打印每个文件的 SHA-256。构建脚本没有 openocd、st-flash、STM32_Programmer_CLI 或任何 program/erase 动作。

12 首次烧录前的强制检查

1. 再次核对 2 MiB 固件备份的大小和 SHA-256;
2. 使用现有供应商固件导出完整 EEPROM, 并对导出文件哈希;
3. 只读记录 Option Bytes、RDP、BOR、boot address、WRP/PCROP;
4. 记录 MCU 型号、UID 的私有清单, 不把 UID 推送到公开仓库;
5. 准备并人工复核恢复命令, 但不要在同一个自动脚本中混合 backup 与 erase;
6. 明确获得用户“允许烧录”的新指令;
7. 首次使用 DIRECT 后端和低速 CLK, 先验证波形;
8. 只有 DIRECT 数据与原固件对齐后, 再测试 DMA 后端。

13 首次上板的测量计划

13.1 阶段 A: 无光学结论的电气验证

1. 示波器通道 1 接 PE2/ST, 通道 2 接 PE3/CLK, 共地;
2. 确认复位后 ST、CLK 均为低;
3. 单帧命令下确认 12 pre-clocks、ST 高宽、dummy/readout 区间;
4. 检查 CLK 占空比、频率与抖动;
5. 检查 USB 枚举、身份回复和停止命令;
6. 不接强光, 读取暗场并检查 ADC 是否饱和或全零。

13.2 阶段 B: 数据语义对照

在同一稳定 LED、光路和积分时间下分别使用供应商固件和新固件采集:

- 暗场均值与标准差;
- 全 288 像素的逐像素差;

- 峰值像素位置；
- 总积分强度；
- 尾部 6 个 flush/trailer 样本。

若光谱整体平移一个像素，应优先检查 dummy clock 和 ADC 触发相位，而不是用软件数组偏移掩盖。

13.3 阶段 C：性能阶梯

建议依次测试 100 kHz、500 kHz、1 MHz、1.5 MHz、2 MHz。每一档记录：

fps, sequence gaps, CRC errors, ADC overrun, dark noise, spectral RMSE.

不能只以 FPS 决定“更快”，因为错误采样、重复帧或丢帧也可能显示很高的 UI 刷新率。

14 性能上限的现实判断

14.1 传感器下限

Hamamatsu 数据手册给出最大 CLK 5 MHz，最短扫描周期约

$$\frac{381}{5 \times 10^6} = 76.2 \mu\text{s}.$$

这说明传感器本身具有远高于当前约 420 FPS 的时间潜力，但实际系统还受到曝光、ADC、USB、主机和光子噪声限制。

14.2 1 kFPS 是否可行

如果一帧为 590 字节，1 kFPS 净载荷为 590 kB/s；V2 帧为 636 字节，对应 636 kB/s。这处于 USB Full-Speed 可尝试的范围，但并无保证。实现条件至少包括：

- 持续流而非逐帧请求；
- 主机串口线程使用足够大的读块；
- 设备不在每帧进行 EEPROM、格式化打印或阻塞延时；
- ADC 与传感器时钟在目标速率下无 overrun；
- 帧序号和 CRC 证明数据真实连续。

如果 CDC 驱动成为瓶颈，下一步是自定义 vendor bulk 接口，而不是修改 C12880 时序。

15 尚未解决的问题

尚未硬件验证

以下问题不能从“已编译”得出答案：

- PE2/PE3 电平极性在实际 PCB 上是否有额外反相器；
- TIM2 CH2 触发相位是否处于模拟视频最佳稳定窗口；
- 2MHz 下 16 位 ADC 的 ENOB、串扰和过载裕量；
- PB6/PB7 是否与 EEPROM 完全一致，以及软件 I2C 的 ACK；
- 供应商主机是否要求身份回复前两个字节的额外语义；
- V2 连续流在具体 Windows USB 栈上的稳定吞吐；
- 校正 EEPROM 中 4096 字节和 0x1068 区域的精确数据结构。

16 结论

现有固件已经被理解到足以建立明确的引脚、时序、ADC、USB 和 EEPROM 契约，并识别出主要性能瓶颈。新的 clean-room 固件把边沿生成、ADC 读出和 USB 发送改造成硬件定时、DMA 与双缓冲流水线，同时保留旧帧并增加可验证的 V2 元数据。它是一个可编译、可审计、可逐级验证的候选固件，但在用户明确授权烧录和完成剩余备份之前，它仍然只应作为构建产物存在。

主要官方资料

- Hamamatsu C12880MA 产品页: <https://www.hamamatsu.com/us/en/product/optical-sensors/spectrometers/mini-spectrometer/C12880MA.html>
- Hamamatsu C12880MA / C16767MA 数据手册: https://www.hamamatsu.com/content/dam/hamamatsu-photonics/sites/documents/99_SALES_LIBRARY/ssd/c12880ma_c16767ma_kacc1226e.pdf
- ST RM0433: https://www.st.com/resource/en/reference_manual/rm0433-stm32h743-753-and-stm32h750-value-line-advanced-arm-based-32-bit-mcus-stmicroelectronics.pdf
- STM32H743 数据手册: <https://www.st.com/resource/en/datasheet/stm32h743vi.pdf>
- STM32H7 HAL: <https://github.com/STMicroelectronics/stm32h7xx-hal-driver>
- ST USB Device Middleware: <https://github.com/STMicroelectronics/stm32-mw-usb-device>